

A HANDS-ON, CODE-FIRST COURSE

LangChain From Zero

Build real GenAI apps with Gemini & local models

PHASE 6 — LOADING & SPLITTING DOCUMENTS

Prerequisites: Phases 1–5 done. `langchain-community` and `pypdf` installed (Phase 0).

Targets: the Document object · PyPDFLoader / TextLoader / WebBaseLoader · RecursiveCharacterTextSplitter

Estimated time for this phase: 60–90 minutes

Where this phase sits

Until now the model only knew what it was trained on. Starting here we feed it **your** data. This phase is the first half of building **RAG**: getting documents *in* (loading) and cutting them into searchable pieces (splitting). Phases 7–8 finish the pipeline.

Ph 0–4 ✓ **Setup** → **Fundamentals** → **Models** → **Prompts** → **Chains**

Ph 5 ✓ **Memory & Conversational Apps** — checkpoint, threads.

Ph 6 **Loading & Splitting Documents** ← you are here
The Document object, loaders for PDF/text/web, chunking with overlap.

Ph 7 **Embeddings & Vector Stores** — turn chunks into searchable vectors (FAISS).

Ph 8 **Building a RAG Application** — retrieve relevant chunks + answer from them.

Ph 9–12 **Tools & Agents · LangGraph · Deployment · Capstone**

Phase 6 · Loading & Splitting Documents

WHAT YOU'LL BE ABLE TO DO AFTER THIS PHASE

Explain why RAG needs your data loaded and chunked; load text files, PDFs, and web pages into LangChain `Document` objects; inspect their content and metadata; and split long documents into overlapping chunks with `RecursiveCharacterTextSplitter`, choosing sensible chunk sizes.

In this phase

1. 6.1 — Why load & split? The RAG motivation
2. 6.2 — The `Document` object: content + metadata
3. 6.3 — Loaders: text, PDF, and web pages
4. 6.4 — Why split at all?
5. 6.5 — `RecursiveCharacterTextSplitter`
6. 6.6 — Choosing chunk size & overlap
7. 6.7 — Common errors & troubleshooting
8. 6.8 — Mini practice exercise
9. Terms introduced this phase

6.1 — Why load & split? The RAG motivation

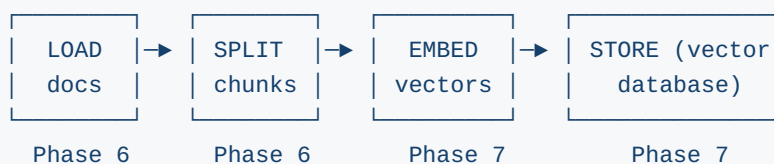
A model only knows what it saw in training. It can't answer questions about your company handbook, a PDF you just downloaded, or yesterday's news. The fix is **RAG**.

TERM: RAG (RETRIEVAL-AUGMENTED GENERATION)

RAG means: when a question comes in, first *retrieve* the most relevant pieces of your own documents, then hand those pieces to the model so it can answer using them. You "augment" the model's reply with retrieved facts — no retraining needed. It's how you build a chatbot over your own data.

RAG has two stages. This phase covers the first two steps of stage one:

INGESTION (do once, ahead of time)



QUERY TIME (every question) → retrieve relevant chunks → feed to model → answer [Phase 8]

So today: turn raw files into clean, bite-sized chunks ready to be made searchable next phase.

6.2 — The `Document` object: content + metadata

Every loader returns a list of **Document** objects — LangChain's standard container for a piece of text plus information about where it came from.

Attribute	What it holds
<code>.page_content</code>	The actual text, as a string.
<code>.metadata</code>	A dict of extra info: source filename, page number, URL, etc. Crucial later for citing <i>where</i> an answer came from.

```
from langchain_core.documents import Document

doc = Document(
    page_content="LangChain gives every model the same interface.",
    metadata={"source": "notes.txt", "author": "Sam"},
)
print(doc.page_content)    # the text
print(doc.metadata)        # {'source': 'notes.txt', 'author': 'Sam'}
```

You rarely build these by hand — loaders create them for you. But knowing the shape matters: chunks, retrieval results, and RAG context are all just `Document` s.

6.3 — Loaders: text, PDF, and web pages

A **document loader** reads a source and returns `Document` s. LangChain has dozens; you'll use three constantly. They all live in `langchain_community.document_loaders` and share a `.load()` method.

NOTE: NO MODEL HERE

Loading and splitting are pure text processing — no LLM is called, so there's no Gemini/Ollama split in this phase. The backends return in Phase 7, where we turn these chunks into embeddings.

Text files — `TextLoader`

First make a sample file so you can run this. Create `sample.txt` with a few paragraphs of any text, then:

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader("sample.txt")    # 1 point at the file
docs = loader.load()                 # 2 -> a list of Document(s)

print(len(docs))                     # 1 (whole file = one Document)
print(docs[0].page_content[:80])     # first 80 chars of the text
print(docs[0].metadata)              # {'source': 'sample.txt'}
```

PDFs — PyPDFLoader

Needs the `pypdf` package (installed in Phase 0). It returns **one Document per page**, with the page number in metadata.

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("handbook.pdf")
pages = loader.load()                # one Document per page

print(len(pages))                    # e.g. 12 (a 12-page PDF)
print(pages[0].metadata)             # {'source': 'handbook.pdf', 'page': 0, ...}
```

Web pages — WebBaseLoader

Fetches a URL and extracts its text. Needs `beautifulsoup4`: run `pip install beautifulsoup4` first.

```
from langchain_community.document_loaders import WebBaseLoader

loader = WebBaseLoader("https://docs.langchain.com/oss/python/langchain/overview")
docs = loader.load()

print(docs[0].metadata["source"])    # the URL
print(len(docs[0].page_content))    # number of characters pulled from the page
```

OTHER LOADERS, SAME PATTERN

`CSVLoader`, `JSONLoader`, `DirectoryLoader` (load a whole folder), and many more all follow the identical "`Loader(...).load()` → list of Documents" shape. Learn one, you've learned them all. There's also `.lazy_load()` which yields documents one at a time — handy for very large files so you don't hold everything in memory at once.

6.4 — Why split at all?

You've got your documents. Why not just feed a whole PDF to the model? Two hard reasons:

- **It won't fit.** A long document blows past the model's context window (Phase 5). You physically can't send it all.
- **Retrieval gets vague.** RAG works by finding the *relevant* bit. If your unit of search is a whole 20-page document, "relevant" is meaningless — you'd retrieve everything. Small chunks let retrieval pinpoint the exact paragraph that answers a question.

TERM: CHUNK / CHUNKING

A **chunk** is a small slice of a document (a few sentences to a few paragraphs). **Chunking** is splitting documents into these slices. Good chunks are big enough to carry a complete idea, small enough to be precisely retrievable. Getting this balance right is one of the biggest levers on RAG quality.

6.5 — RecursiveCharacterTextSplitter

The go-to splitter. "Recursive" means it tries to split on natural boundaries first — paragraphs (`\n\n`), then lines (`\n`), then sentences, then words — only breaking mid-word as a last resort. This keeps related text together.

⚠ IMPORT PATH CHANGED

Splitters now live in their own package: `from langchain_text_splitters import RecursiveCharacterTextSplitter`. Old tutorials use `from langchain.text_splitter import ...` — that path is legacy. Use `langchain_text_splitters`.

```
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter # 1

docs = TextLoader("sample.txt").load()

splitter = RecursiveCharacterTextSplitter( # 2
    chunk_size=500, # max characters per chunk
    chunk_overlap=50, # characters shared between neighbours
    add_start_index=True, # record where each chunk began
)

chunks = splitter.split_documents(docs) # 3 -> list of smaller Documents

print(len(chunks)) # many small chunks
print(chunks[0].page_content) # the first chunk's text
print(chunks[0].metadata) # source + start_index, carried over
```

Line	Explanation
1	Import from the dedicated <code>langchain_text_splitters</code> package.
2	Configure the splitter. <code>chunk_size</code> caps each chunk's length (in characters by default). <code>chunk_overlap</code> repeats a little text across the boundary. <code>add_start_index=True</code> stores each chunk's start position in metadata.
3	<code>split_documents</code> takes Documents and returns smaller Documents — metadata is preserved , so each chunk still knows its source and page. (Use <code>split_text(a_string)</code> if you have a raw string instead of Documents.)

TERM: CHUNK OVERLAP

Overlap repeats the last few characters of one chunk at the start of the next. Why? So an idea that straddles a boundary isn't cut in half and lost. Without overlap, "...the refund policy is" | "30 days from purchase" splits the answer across two chunks and retrieval may grab only one. A small overlap (10–20% of chunk size) stitches the seam.

no overlap:	[....chunk A....][....chunk B....]	seam can cut an idea
with overlap:	[....chunk A....] [....chunk B....]	B repeats A's tail

6.6 — Choosing chunk size & overlap

There's no universal "right" number — it depends on your content and model. Sensible starting points:

Content type	chunk_size (chars)	chunk_overlap	Why
Dense prose / articles	800–1,200	100–200	Paragraph-sized ideas stay intact.
Q&A, FAQs, short notes	300–600	30–80	Precise retrieval of small facts.
Code / technical docs	500–1,000	50–150	Keep functions/sections together.
Legal / reference	1,000–1,500	150–250	Clauses need surrounding context.

HOW TO TUNE IN PRACTICE

Start around `chunk_size=1000`, `chunk_overlap=150`, build the RAG pipeline (Phases 7–8), then look at what gets retrieved for real questions. If answers miss context, increase size/overlap; if retrieval is noisy and grabs unrelated text, decrease size. Chunking is iterative — you'll revisit it. Measuring length in *tokens* rather than characters is also possible via a `length_function`, but characters are fine to start.

HOW TO TUNE IN PRACTICE

Start around `chunk_size=1000, chunk_overlap=150`, build the RAG pipeline (Phases 7–8), then look at what gets retrieved for real questions. If answers miss context, increase size/overlap; if retrieval is noisy and grabs unrelated text, decrease size. Chunking is iterative — you'll revisit it. Measuring length in *tokens* rather than characters is also possible via a `length_function`, but characters are fine to start.

6.7 — Common errors & troubleshooting

Symptom	Likely cause & fix
<code>ModuleNotFoundError: pypdf</code> when loading a PDF	<code>pip install pypdf</code> (was in the Phase 0 list — check your venv is active).
<code>ModuleNotFoundError: bs4</code> / WebBaseLoader fails	Install BeautifulSoup: <code>pip install beautifulsoup4</code> .
<code>ImportError</code> for <code>RecursiveCharacterTextSplitter</code>	Wrong path. Use <code>from langchain_text_splitters import ...</code> , not <code>langchain.text_splitter</code> .
<code>FileNotFoundError</code> from a loader	The path is relative to where you run the script. Use an absolute path or run from the file's folder.
PDF loads but <code>page_content</code> is empty	The PDF is scanned images, not text. You need OCR (e.g. an OCR-capable loader) — plain <code>PyPDFLoader</code> only reads embedded text.
Chunks cut awkwardly mid-sentence	Increase <code>chunk_overlap</code> , or raise <code>chunk_size</code> . Recursive splitting already prefers paragraph/line boundaries.
Way too many tiny chunks	<code>chunk_size</code> is too small for your content. Raise it (see §6.6).

6.8 — Mini practice exercise

YOUR TURN (DO THIS BEFORE PHASE 7)

Task: Write `ingest.py` that (1) loads a document of your choice — a real PDF you have, or a web page via `WebBaseLoader` ; (2) prints how many Documents loaded and the metadata of the first; (3) splits everything with `RecursiveCharacterTextSplitter` ; (4) prints the total chunk count and the length of the first three chunks; and (5) prints the `metadata` of the first chunk to confirm `source` (and `start_index`) survived the split. Then experiment: re-run with two different `chunk_size` values and observe how the chunk count changes.

Why: this is the exact ingestion code you'll reuse — unchanged — in Phases 7 and 8. Getting comfortable inspecting chunks now means RAG won't feel like a black box later.

Hints (not the full answer):

- Loader → `.load()` → `split_documents(...)` . Three lines of real work.
- Chunk length is `len(chunk.page_content)` ; loop over `chunks[:3]` .
- Smaller `chunk_size` ⇒ more chunks. Confirm the relationship by trying 300 vs 1,000.
- If you used a multi-page PDF, peek at a chunk's `metadata["page"]` — retrieval will use this to cite sources in Phase 8.

Terms introduced this phase

Term	One-line meaning
RAG	Retrieve relevant pieces of your data, then have the model answer using them.
Ingestion	The one-time load → split → embed → store pipeline that prepares your data.
Document	LangChain's text container: <code>.page_content</code> + <code>.metadata</code> .
Document loader	Reads a source (file/URL) and returns a list of Documents via <code>.load()</code> .
PyPDFLoader / TextLoader / WebBaseLoader	Loaders for PDFs (one Doc per page), text files, and web pages.
metadata	Per-document info (source, page, URL) — used later to cite answers.
Chunk / chunking	A small slice of a document / the act of splitting into slices.
RecursiveCharacterTextSplitter	Splits on natural boundaries (paragraph → line → word) into sized chunks.
chunk_size	Maximum length of each chunk (characters by default).
chunk_overlap	Text repeated across chunk boundaries so ideas aren't cut in half.

PHASE 6 COMPLETE

You can load text, PDFs, and web pages into **Document** objects, inspect their content and metadata, and split them into overlapping chunks with sensible sizing. That's the raw material of RAG ready to go. **Next — Phase 7: Embeddings & Vector Stores**, where the backends return: you'll turn these chunks into numerical *embeddings* (with both Gemini and a local Ollama model), store them in a FAISS vector database, and search by meaning rather than keywords.